TUXOR: Operator-Based Dual-Hash Authentication Algorithm with User-Embedded Function Selection

Bernardo Sanchez Gutierrez
tuxor.max@gmail.com

April 2026

Abstract

We present TUXOR, a novel authentication algorithm that introduces a fundamentally new concept in credential hashing: user-defined operation selection embedded within the credentials themselves. Unlike traditional schemes where the combining function is public and fixed, TUXOR allows users to embed mathematical and bitwise operator symbols within both their identity and secret inputs. These operators determine how independently hashed blocks of the two inputs are combined, making the operation selection itself part of the authentication secret. Only the final computed value—the *tuxor*—is stored; neither the identity nor the secret is persisted in any form. The algorithm further introduces an inclusion modifier (`@`) that controls whether operator symbols are incorporated into the hashed text, yielding four distinct modes from identical base credentials. Combined with a standard key derivation function (Argon2id or scrypt), TUXOR provides a confusion layer that significantly increases the search space for brute-force attacks. We provide reference implementations in PHP, JavaScript, and Python, with cross-platform test vectors demonstrating identical outputs.

Keywords: authentication, password hashing, operator-based cryptography, dual-hash, key derivation

1 Introduction

Password authentication remains the dominant mechanism for user verification in information systems. The standard approach involves hashing the user's password with a one-way function and storing the hash for later comparison. Modern best practices employ key derivation functions (KDFs) such as bcrypt [1], scrypt [2], or Argon2 [3] that introduce computational cost to resist brute-force attacks.

However, all existing schemes share a common architectural property: the *combining function* used to process credentials is publicly known and fixed. Whether a system uses HMAC's XOR-based construction, bcrypt's Blowfish rounds, or Argon2's memory-hard passes, the algorithmic steps are deterministic given the input. An attacker who obtains a database dump knows exactly which function produced each stored hash and can focus entirely on guessing the input.

TUXOR challenges this assumption by making the combining function itself part of the user's secret. Users embed mathematical operator symbols (`+`, `-`, `*`, `%`, `^`, `&`, `|`, `<`, `>`, `#`) within their identity and secret strings. These operators are extracted during parsing and used to determine how independently hashed blocks of the two inputs are combined. The result is that an attacker must guess not only the identity and secret, but also which operations were applied—and in what order.

This paper formalizes the TUXOR algorithm, analyzes its security properties, compares it with related work, and presents reference implementations with cross-platform test vectors.

2 Related Work

2.1 SXR Algorithm

Polpong [5] proposed SXR, which splits a single password hash into two parts based on a ratio derived from the credentials, then applies XOR iteratively between the halves. While SXR shares the concept of block-level manipulation with input-derived parameters, it operates on a *single* hash using a *fixed* operation (XOR).

2.2 LG Algorithm

Anbari et al. [6] introduced binary randomization through logical gates (AND, OR, XOR, NAND, NOR, NOT) applied to the binary representation of username and password before hashing. However, the gate sequence is a *fixed system parameter*, not selected by the user.

2.3 HMAC

HMAC [4] combines a secret key with a message using two passes of a hash function with XOR operations against fixed padding constants (ipad/opad). The combining operations are public and invariant.

2.4 Hash Combiners

Fischlin et al. [7] studied robust multi-property hash combiners that merge two hash functions into a single secure construction. The combining function is always fixed and public.

2.5 Key Derivation Functions

Modern KDFs—bcrypt [1], scrypt [2], and Argon2 [3]—focus on computational and memory cost to resist brute-force attacks. They process a single input (password + salt) through a fixed algorithm. TUXOR is designed to complement, not replace, these functions.

2.6 Novelty of TUXOR

To our knowledge, no published algorithm treats the combining function itself as a user-defined secret embedded within the credential inputs. This represents TUXOR's primary contribution.

3 Algorithm Specification

3.1 Definitions

- **Identity** (I_{in}): First input string containing at least one operator symbol.
- **Secret** (S_{in}): Second input string containing at least one operator symbol.
- **Operator**: A symbol from the set $\Omega = \{+, -, *, \%, \wedge, \&, |, <, >, \#\}$.
- **Tuxor**: The final 256-bit hash—the only value stored.
- **Block**: A 64-bit unsigned integer derived from 16 hexadecimal characters.

3.2 Valid Operators

Table 1 defines the ten supported operators.

Table 1: TUXOR Operators

Symbol	Operation	Type	Definition
`+`	Addition	Arithmetic	$(I + S) \bmod 2^{64}$
`-`	Subtraction	Arithmetic	$(I - S + 2^{64}) \bmod 2^{64}$
`*`	Multiplication	Arithmetic	$(I \times S) \bmod 2^{64}$
`%`	Modulo	Arithmetic	$I \bmod S$ (if $S = 0$, use $S = 1$)
`^`	XOR	Bitwise	$I \oplus S$
`&`	AND	Bitwise	$I \wedge S$
`|`	OR	Bitwise	$I \vee S$
`<`	Left Rotate	Shift	Rotate I left by ($S \bmod 64$) bits
`>`	Right Rotate	Shift	Rotate I right by ($S \bmod 64$) bits
`#`	Re-Hash	Special	hex_to_int(SHA-256($I \| S$)[0..15])

3.3 Input Parsing

Both inputs follow the format: [operators_prefix] clean_text [operators_suffix]

Operators are extracted from the beginning and end of each string. The remaining text constitutes the "clean text" that will be hashed.

3.3.1 Inclusion Modifier `@`

The `@` symbol acts as a modifier controlling whether operators are included in the hashed text:

Table 2: Inclusion Modifier Modes

Modifier	Position	Mode	Effect
(none)	—	`none`	Operators excluded from clean text
`@`	Start	`prefix`	Prefix operators included in clean text
`@`	End	`suffix`	Suffix operators included in clean text
`@@`	Start or End	`all`	All operators included in clean text

3.4 Algorithm Steps

Algorithm 1 TUXOR Computation

Require: Identity I_{in}, Secret S_{in}
Ensure: 64-character hexadecimal tuxor

```
Parse I_in and S_in: extract operators and clean text
ops ← I.prefix‖I.suffix‖S.prefix‖S.suffix
H_I ← SHA-256(I.clean)                          ▷ 64 hex characters
H_S ← SHA-256(S.clean)                          ▷ 64 hex characters
Split H_I into blocks [I_0, I_1, I_2, I_3]      ▷ 16 hex chars each
Split H_S into blocks [S_0, S_1, S_2, S_3]
for n = 0 to 3 do
    op ← ops[n mod |ops|]
    R_n ← apply(op, I_n, S_n)                   ▷ See Table 1
end for
return SHA-256(hex(R_0)‖hex(R_1)‖hex(R_2)‖hex(R_3))
```

3.5 Secure Mode: Salt and Key Derivation

For production use, TUXOR is combined with a standard KDF:

Algorithm 2 TUXOR Secure Mode

Require: Identity I_{in}, Secret S_{in}, Cost c
Ensure: Hardened tuxor, salt

```
tuxor_raw ← TUXOR(I_in, S_in)                                  ▷ Algorithm 1
salt ← random(16 bytes)
input ← tuxor_raw‖':'‖salt
tuxor_final ← Argon2id(input, salt, memory = 2^c, t = 4, p = 2)
Store: tuxor_final, salt, c
```

This two-layer approach provides:

- **Confusion layer** (TUXOR): operator-based secret combination
- **Resistance layer** (Argon2id/scrypt): computational and memory cost

4 Security Analysis

4.1 Threat Model

We consider an attacker who has obtained a full database dump containing stored tuxor values, salts, and cost parameters. The attacker has knowledge of the TUXOR algorithm specification.

4.2 Attack Resistance

4.2.1 Brute-Force Search Space

An attacker must guess three independent components:

$$\text{Search Space} = |T_I| \times |T_S| \times |O| \times |M| \tag{1}$$

where $|T_I|$ and $|T_S|$ are the text spaces for identity and secret, $|O|$ is the operator combination space, and $|M| = 4$ is the inclusion modifier space.

For the operator component alone:

$$|O| = \sum_{k=1}^{n} |\Omega|^k = \sum_{k=1}^{n} 10^k \tag{2}$$

With $n = 4$ operator positions (practical maximum), $|O| = 11{,}110$ combinations, contributing $\sim$13.4 bits of entropy. Combined with the modifier: $|O| \times |M| = 44{,}440$ ($\sim$15.4 bits).

4.2.2 Rainbow Tables

Rainbow tables are precomputed mappings from inputs to hashes. TUXOR resists these attacks because:

1. The tuxor is not a direct hash of any single known input
2. The operator combination creates a vast input space
3. In secure mode, the unique salt per user eliminates rainbow table applicability entirely

4.2.3 Database Compromise

A compromised database reveals only:

- The tuxor value (irreversible)
- The salt and cost parameter (in secure mode)
- The display identity (not used for authentication)

No username hash, password hash, or operator information is stored.

4.3 Non-Reversibility

The tuxor undergoes two layers of SHA-256 (block operations followed by final hash), making it computationally infeasible to decompose into its original components. The operator applied to each block pair is unknown to the attacker, creating 10^k possible decomposition paths per block.

4.4 Comparison with Existing Schemes

Table 3: Comparison with Related Schemes

Property	HMAC	SXR	LG	bcrypt	**TUXOR**
Two independent hashes	No	No	No	No	**Yes**
Block-level combination	No	Yes	No	No	**Yes**
User-selected operation	No	No	No	No	**Yes**
Operation as secret	No	No	No	No	**Yes**
No credential storage	No	No	No	No	**Yes**
Memory-hard	No	No	No	Yes*	Yes†

*bcrypt is CPU-hard. †Via Argon2id/scrypt in secure mode.

5 Implementation

5.1 Reference Implementations

We provide implementations in three languages, all producing identical output for the same inputs:

- **PHP**: Native 64-bit arithmetic with split 32-bit operations (no extensions required). Secure mode uses `password_hash()` with `PASSWORD_ARGON2ID`.
- **JavaScript**: BigInt for 64-bit operations (Node.js 16+ and modern browsers). Secure mode uses `crypto.scrypt()`.
- **Python**: Native arbitrary-precision integers. Secure mode uses `hashlib.scrypt()`.

5.2 Test Vector

All implementations must produce the following output for conformance:

```
Identity: "+tuxor"
Secret:   "*algorithm#"
Operators: [+, *, #]

Tuxor: 663b623d1f5f78b197cfe54fbdbb47dcb679c8842e0bb138
       d90e001aaa50fdb8
```

5.3 Test Results

Table 4: Test Suite Results

Implementation	Tests	Status
PHP (Argon2id)	35	All passed
Python (scrypt)	34	All passed
JavaScript (scrypt)	34	All passed

6 Account Recovery

Since the tuxor is non-reversible and no individual credential is stored, account recovery requires a full credential reset. Each user registers a *recovery contact*—either an email address or phone number. The system auto-detects the type based on format (presence of `@` for email, digits only for phone).

Upon recovery, the user defines entirely new identity and secret credentials, and a new tuxor is computed and stored, replacing the previous one.

7 Discussion

7.1 Limitations

1. **User complexity**: The requirement for operators in both inputs increases cognitive load compared to traditional passwords.
2. **Recovery**: Forgetting the operator pattern results in complete credential loss, mitigated by the recovery mechanism.
3. **Custom scheme**: TUXOR has not undergone formal cryptographic audit. We recommend its use as a complementary layer rather than a standalone replacement.

7.2 Recommended Use Cases

- **Internal/corporate systems** with trained users
- **Machine-to-machine authentication** (APIs, microservices)
- **Defense-in-depth layer** combined with standard KDFs
- **High-security environments** where additional complexity is acceptable

7.3 Future Work

- Formal security proof under standard cryptographic assumptions
- Performance benchmarking across platforms and hardware
- Integration libraries for common frameworks (Laravel, Django, Express)
- Hardware-accelerated implementation in C/Rust
- User study on usability and memorability of operator-embedded credentials

8 Conclusion

TUXOR introduces a novel concept in authentication: embedding the combining operation within the user's credentials as a secret. By requiring operator symbols in both identity and secret inputs, the algorithm creates a three-factor knowledge requirement (identity + secret + operations) that has no precedent in published cryptographic literature.

The inclusion modifier `@` further multiplies the search space by controlling whether operators are hashed as part of the clean text, yielding four distinct modes. Combined with Argon2id or scrypt as a hardening layer, TUXOR provides both a confusion layer (unknown operations) and a resistance layer (computational cost).

Reference implementations in PHP, JavaScript, and Python demonstrate cross-platform consistency, with all implementations producing identical tuxor values for the same inputs.

The source code and full specification are available at `https://github.com/tuxormax/tuxores` under the GPL-3.0 license.

References

[1] N. Provos and D. Mazières, "A future-adaptable password scheme," in *Proceedings of the USENIX Annual Technical Conference*, 1999.

[2] C. Percival, "Stronger key derivation via sequential memory-hard functions," in *BSDCan*, 2009.

[3] A. Biryukov, D. Dinu, and D. Khovratovich, "Argon2: New generation of memory-hard functions for password hashing and other applications," in *IEEE European Symposium on Security and Privacy*, 2016.

[4] M. Bellare, R. Canetti, and H. Krawczyk, "Keying hash functions for message authentication," in *CRYPTO'96*, Springer, 1996.

[5] P. Polpong, "SXR: A new password hashing algorithm based on XOR and rotation," *International Journal of Electrical and Computer Engineering (IJECE)*, vol. 11, no. 6, 2021.

[6] M. Anbari et al., "Enhancing the resistance of password hashing using binary randomization through logical gates," *International Journal of Electrical and Computer Engineering (IJECE)*, 2024.

[7] M. Fischlin and A. Lehmann, "Multi-property preserving combiners for hash functions," *Journal of Cryptology*, vol. 26, 2013.